Homework #3 (You need to complete modules 5, 6, and 7 to complete this homework)

Start Assignment

- Due Saturday by 11:59pm
- Points 60
- Submitting a file upload
- Available Jan 13 at 12am - Mar 7 at 11:59pm

For this Homework assignment, use this Submission template provided at the bottom.

High coupling in a Java program means that the classes in the program are highly dependent on each other. High coupling is generally considered a bad practice as it makes the code less modular, harder to maintain, and makes changes. In this homework, you will work on three simple programs that have high coupling to make them less coupling.

Problem 1 [20 Pts] Removing high cohesion using design patterns

Consider the following code given. In this code:

1. The Car class directly creates instances of Engine, Transmission, and Wheels within its constructor. This creates a high degree of dependency between Car and these components.
2. Car cannot function without these specific components, and any change to Engine, Transmission, or Wheels will likely require changes in Car as well.
3. If we wanted to replace Engine with a different implementation, we'd have to modify the car class also.

You can access the following code here: **Question 1 Code (https://canvas.asu.edu/courses/248379/files/123147477?wrap=1) (https://canvas.asu.edu/courses/248379/files/123147477/download?download_frd=1)**

```
class Engine {

   public void start() {

      System.out.println("Engine started");

   }

}

class Transmission {

   public void shift() {
```

```
        System.out.println("Transmission shifted");
    }
}
class Wheels {
    public void roll() {
        System.out.println("Wheels are rolling");
    }
}
class Car {
    private Engine engine;
    private Transmission transmission;
    private Wheels wheels;

    public Car() {
        engine = new Engine();
        transmission = new Transmission();
        wheels = new Wheels();
    }
    public void drive() {
        engine.start();
        transmission.shift();
        wheels.roll();
        System.out.println("Car is driving");
    }
}
public class Main {
    public static void main(String[] args) {
```

```
        Car car = new Car();

        car.drive();

    }

}
```

This high coupling can be reduced by using the design of the Factory pattern. Rewrite the above program using the factory pattern.

Problem 2 [20 Pts] Removing content coupling:

Consider the following code given. In this code, In this example:

1. The Car class directly accesses and modifies the fuelLevel field of the Engine class. This is an example of content coupling, as Car is tightly coupled to the internal implementation of Engine.
2. Any changes to the internal structure of Engine (e.g., renaming fuelLevel or changing its type) will require changes in the Car class.
3. This tight coupling makes the code difficult to maintain and less modular.

You can access the following code here: **Question 2 Code (https://canvas.asu.edu/courses/248379/files/123147642?wrap=1)** **(https://canvas.asu.edu/courses/248379/files/123147642/download?download_frd=1)**

```
class Engine {

    public int fuelLevel;

    public boolean isRunning;



    public Engine() {

        this.fuelLevel = 100;

        this.isRunning = false;

    }

    public void start() {

        if (fuelLevel > 0) {

            isRunning = true;

            System.out.println("Engine started");

        } else {
```

```
            System.out.println("Not enough fuel to start the engine");
        }
    }
}
class Car {
    private Engine engine;


    public Car() {
        engine = new Engine();
    }
    public void drive() {
        // Directly accessing and modifying the internal state of the Engine
        engine.fuelLevel -= 10; // Content coupling
        if (!engine.isRunning) {
            engine.start();
        }
        if (engine.isRunning) {
         System.out.println("Car is driving with fuel level: " + engine.fuelLevel);
        }
    }
}
public class Main {
    public static void main(String[] args) {
                 Car car = new Car();
                car.drive();
                car.drive();
    }
```

```
}
```

To avoid content coupling, encapsulation should be enforced by making fields private and providing public methods to manipulate the state. Rewrite the above program to eliminate content coupling.

Problem 3 [20 Pts] Redesigning code to improve cohesion

Lack of Cohesion of Methods (LCOM) is a metric used to measure how well the methods of a class are related to each other. A high LCOM value indicates low cohesion, meaning that the class has methods that are not related to each other or that operate on disjoint sets of instance variables.

LCOM4, a variant of the LCOM metric, focuses on the lack of cohesion based on the disjoint sets of instance variables used by the methods. A class with an LCOM4 value greater than 1 indicates a lack of cohesion, as the methods do not share common attributes.

Here's an example of a Java class that demonstrates a lack of cohesion with an LCOM4 value greater than 1:

In this example:

1. The Employee class has methods that operate on disjoint sets of instance variables:
   - printEmployeeDetails() operates on name and id.
   - giveRaise() operates on salary.
   - changeDepartment() operates on department.
   - updateAddress() operates on address.
2. These methods do not share common attributes, indicating a lack of cohesion.
3. The LCOM4 value for this class would be greater than 1, as the methods do not share enough common attributes.

You can access the following code here: **Question 3 Code (https://canvas.asu.edu/courses/248379/files/123147641?wrap=1)** **(https://canvas.asu.edu/courses/248379/files/123147641/download?download_frd=1)**

```
public class Employee {

    private String name;

    private int id;

    private double salary;

    private String department;

    private String address;
```

```
    // Method related to name and id
    public void printEmployeeDetails() {
        System.out.println("Name: " + name);
        System.out.println("ID: " + id);
    }

    // Method related to salary
    public void giveRaise(double percent) {
        salary += salary * percent / 100;
    }

    // Method related to department
    public void changeDepartment(String newDepartment) {
        department = newDepartment;
    }

    // Method related to address
    public void updateAddress(String newAddress) {
        address = newAddress;
    }
}
public class Main {
    public static void main(String[] args) {
        Employee emp = new Employee();

        // Using various unrelated methods from Employee class
        emp.printEmployeeDetails();
```

```
        emp.giveRaise(10);

        emp.changeDepartment("HR");

        emp.updateAddress("123 Main St");

    }

}
```

A more cohesive design would split these responsibilities into separate classes or group methods that operate on common attributes together. For example, methods related to employee details, salary, department, and address could be separated into different classes or grouped in a more cohesive manner. Rewrite the above program to increase the cohesion so that LCOM4 value is not greater than one .

Submission

Follow these steps for assignment completion:

1. Use the **homework #3 submission template (https://canvas.asu.edu/courses/248379/files/123147569?wrap=1)**

 and save your file as *LastnameFirstInitial_HW#3* (Ex. SmithR_HW#2)

2. On the top of this page select Start Assignment

3. In the 'File Upload' section, select Upload File

4. Use the Choose File SmithR_HW#1.docx option to locate and upload your saved file.

5. Be sure to hit Submit Assignment to finish submission

HW #3 Grading

Criteria	Ratings				Pts
Correct Application of the Factory or other suitable Pattern/Technique	**10 pts** **Full Marks** The factory pattern or the selected design pattern/technique was implemented correctly by decoupling object creation fully from the car class.	**8 pts** **Partial Credit** Solution is mostly correct, but one component is still created directly within the Car class.	**5 pts** **Partial Credit** Selected technique is partially applied and more than one component is still tightly coupled.	**0 pts** **No credit** No suitable design pattern/technique applied , or it's implemented incorrectly.	10 pts
Reduction of High Coupling	**5 pts** **Full Marks** Dependencies between classes are minimized, with clear decoupling of components.	**4 pts** **Partial Credit** Dependencies are reduced significantly, but the solution still contains some minor coupling.	**2 pts** **Partial Credit** A slight reduction in dependencies, but much of the code remains tightly coupled.	**0 pts** **No Marks** No decoupling applied.	5 pts
Code Correctness and Functionality	**5 pts** **Full Marks** Code runs correctly.	**4 pts** **Partial Credit** Code mostly works, but contains small logical or functional issues.	**2 pts** **Partial Credit** Code compiles, but has major issues in functionality.	**0 pts** **No Marks** Code does not compile.	5 pts
Encapsulation and Elimination of Content Coupling	**10 pts** **Full Marks** All fields in the Engine class are made private, and the Car class interacts with the Engine using public methods only.	**8 pts** **Partial Credit** Most fields are encapsulated, but some content coupling still remains.	**5 pts** **Partial Credit** Partial encapsulation applied, but major content coupling issues remain.	**0 pts** **No Marks** No encapsulated fields.	10 pts
Correct Usage of Getter/Setter	**5 pts** **Full Marks**	**4 pts** **Partial Credit**	**2 pts** **Partial Credit**	**0 pts** **No Marks**	5 pts

Criteria	Ratings				Pts
Methods or Access Methods	Getter/setter used correctly to access and modify the engine's state.	Most access to fields is through getter/setter methods, but some direct field modifications still occur.	Some getter/setter methods implemented, but significant direct field access remains.	No Getter/setter methods were used.	
Code Correctness and Functionality	**5 pts** **Full Marks** Code compiles.	**4 pts** **Partial Credit** Code runs as expected, but contains minor issues with encapsulation or functionality.	**2 pts** **Partial Credit** Code compiles but major functionality is incomplete.	**0 pts** **No Marks** Code does not compile or perform correctly.	5 pts
Improvement of Class Cohesion	**10 pts** **Full Marks** Class responsibilities are well separated, and methods that operate on related fields are grouped, resulting in an LCOM4 ≤ 1.	**8 pts** **Partial Credit** Cohesion is significantly improved, but there are minor unrelated methods within the class.	**5 pts** **Partial Credit** Some improvements in cohesion, but several methods still operate on disjoint attributes.	**0 pts** **No Marks** No improvements in cohesion.	10 pts
Appropriate Class Design	**5 pts** **Full Marks** Classes have well-defined, logical responsibilities, with methods grouped appropriately into separate classes or cohesive groupings.	**4 pts** **Partial Credit** Mostly logical class design, but some responsibilities could be better separated or grouped.	**2 pts** **Partial Credit** Some improvement in design, but classes still mix unrelated responsibilities.	**0 pts** **No Marks** Class design is inappropriate.	5 pts

Criteria	Ratings				Pts
Code Correctness and Functionality	**5 pts** **Full Marks** Code compiles and runs correctly.	**4 pts** **Partial Credit** Code works mostly as intended, but contains minor issues with class interactions or functionality.	**2 pts** **Partial Credit** Code compiles, but significant functionality issues are present.	**0 pts** **No Marks** Code does not compile or perform correctly.	5 pts
					Total Points: 60